



LAB 2 REPORT



SNOW BOARDER

2D Physics-Based Arcade Game Development

Student Name: NGUYỄN CÔNG NHẤT

Student ID: QE190017

Class/ Course: SE19B.NET - PRU213

Submission: Lab 2 – Unity Game Development

Table of Contents

1. Project Overview	3
1.1. Introduction	3
1.2. Game Objective	3
2. Pre-Lab Setup and Environment Configuration	3
2.1. Display and Camera Configuration	3
2.2. Tag System Initialization	4
2.3. Project Directory Organization	4
3. Scene Management & User Interface (UI)	4
3.1. Build Settings and Scene Flow	4
3.2. Main Menu and In-Game HUD	5
4. Core Mechanics & Physics Scripting	5
4.1. Movement and Transform Logic	5
4.2. Physics and Collision Detection	6
4.3. Jump Mechanic	6
5. Creative Implementations (Polish & Game Feel)	7
5.1. Scoring and Combo System	7
5.2. Dynamic Weather Effects	7
5.3. High Score Persistence & Leaderboard	7
5.4. Audio Feedback System	8
6. Challenges Faced & Accommodations	8
7. Feature Completion Summary	8
8. Conclusion	9

1. Project Overview

1.1. Introduction

"Snow Boarder" is a 2D arcade sports game developed using the Unity Engine (C# scripting). The project demonstrates the advanced application of Unity's 2D physics engine to create realistic slope mechanics, dynamic player controls, and an engaging scoring system based on both collectible point gems and kinesthetic physics tricks (aerial spins). The game consists of two playable levels with progressively challenging terrain designed using Unity's Sprite Shape system.

1.2. Game Objective

The player controls a snowboarder navigating through hazardous downhill slopes. Primary objectives include:

- Maintain speed and momentum down the slope using SurfaceEffector2D-driven physics.
- Avoid crashing — landing head-first on the ground or falling into a FallZone triggers Game Over.
- Collect PointGems scattered across the terrain for bonus score.
- Perform aerial 360° spin tricks while airborne to earn trick points and build combo multipliers.
- Reach the Finish Line flag to complete the level and advance to the next.

2. Pre-Lab Setup and Environment Configuration

2.1. Display and Camera Configuration

The project targets a fixed 16:9 aspect ratio to maintain a consistent "Crisp 2D" vector aesthetic. The Main Camera uses Orthographic projection. A Cinemachine Virtual Camera (VC Follow Camera) is used for dynamic player-following, providing smooth damping as the snowboarder descends the slope. The camera maintains a vertical offset to keep upcoming obstacles visible.

2.2. Tag System Initialization

The following Tag system manages the collision and trigger matrix:

Tag	Assigned To	Purpose
Player	Barry (root GameObject)	Identified by FinishLine, Point triggers
Ground	Sprite Shape terrain objects	Validates landings; crash if head touches
FallZone	FallDetector trigger volume	Instant Game Over when player falls off map

Obstacle	Rock / Tree GameObjects	Alternative identification for hazards
Collectible	PointGem prefab instances	Score pickup items

2.3. Project Directory Organization

The Assets directory is organized for maintainability and scalability:

Folder	Contents
Assets/Scripts/Player/	PlayerController.cs, CrashDetector.cs, FinishLine.cs, DustParticles.cs
Assets/Scripts/Managers/	GameManager.cs, LevelManager.cs, LeaderboardManager.cs, MainCamera.cs, Point.cs
Assets/Scripts/Score/	HighestScore.cs, HighScoreManager.cs
Assets/Scenes/	Main Menu.unity, Level1.unity, Level2.unity
Assets/Prefabs/	GameManager.prefab, PointGem.prefab, Finish Line.prefab, Snow Generator.prefab, Canvas.prefab
Assets/Art/Image/	Sprites: Boarder_Top/Bottom, Snow tiles, Trees, Rocks, Clouds, Backgrounds
Assets/Audio/	BGM tracks (MP3), SFX: Crash SFX, Finish SFX, Collect sound
Assets/Animation/	Flag animation sprites

3. Scene Management & User Interface (UI)

3.1. Build Settings and Scene Flow

UnityEngine.SceneManagement is used for all scene transitions. Scenes are registered in Build Settings with the following index progression:

Index	Scene	Role
0	Main Menu.unity	Title screen with Play, Options, High Score, Quit buttons
1	Level1.unity	First playable level – introductory slopes and obstacles
2	Level2.unity	Second level – more complex terrain and obstacle density

Game Over and Win states are handled via UI overlay panels (Canvas → Panels → GameOver / Win) inside the gameplay scene, avoiding unnecessary scene reloads and preserving runtime state.

3.2. Main Menu and In-Game HUD

Main Menu features:

- Play button — loads Level 1 via SceneManager.LoadSceneAsync(1).
- Options button — placeholder for future audio/difficulty settings.
- High Score button — displays top-5 scores stored in PlayerPrefs.
- Quit button — calls Application.Quit().

In-Game HUD (Canvas, always-on) displays four real-time metrics using TextMeshPro:

HUD Element	Script Source	Description
Score	PlayerController.score	Total accumulated points (gems + tricks)
Speed	rb.linearVelocity.magnitude	Current movement speed in m/s
Distance	Vector2.Distance(start, current)	Total distance traveled from spawn in meters
Combo	comboMultiplier	Active spin combo multiplier (x1 ... xN)

Design decision: Player "lives" were intentionally omitted to favour an arcade retry loop. A crash results in immediate Game Over, encouraging mastery of landing technique.

4. Core Mechanics & Physics Scripting

4.1. Movement and Transform Logic

Player movement is entirely physics-driven, implemented in PlayerController.cs (attached to the Barry root GameObject):

- Rigidbody2D — carries mass, gravity scale, and linear/angular drag for realistic inertia.
- SurfaceEffector2D (on terrain) — applies a continuous tangential surface force. Player input modifies se.speed dynamically: W/Up = boostSpeed, S/Down = slowSpeed, default = baseSpeed.
- AddTorque (A/D or Arrow keys) — rotates the player for balance and aerial spin execution. Clockwise or counter-clockwise torque is applied each frame the key is held.

Key serialized parameters configurable in the Unity Inspector:

Field	Default Use	Effect
TorqueAmount	Rotation force	Controls how fast the player can rotate/spin
boostSpeed	W key speed	SurfaceEffector speed when accelerating
baseSpeed	Normal speed	Default downhill surface speed
slowSpeed	S key speed	SurfaceEffector speed when braking
jumpForce	Space key	Initial Y-velocity applied on jump

4.2. Physics and Collision Detection

CrashDetector.cs (on Barry) handles two distinct crash triggers:

```
OnTriggerEnter2D → tag == "Ground" || tag == "FallZone" → Crash()
```

The head/body trigger collider detects when the player's upper body enters the ground surface (meaning the player landed upside-down) or falls into a FallZone volume. Rock obstacles use a solid BoxCollider2D with a bouncy PhysicsMaterial2D (bounciness = 0.7, friction = 0) to deflect the player without causing an immediate crash, preserving natural arcade feel.

FinishLine.cs handles level completion:

```
OnTriggerEnter2D → tag == "Player" → finishEffect.Play() → ShowWin()
```

Upon triggering the Finish Line, the player controller is disabled, a particle effect plays, and after a short delay the Win panel activates and the high score is saved.

4.3. Jump Mechanic

A jump system (Space key) was implemented to allow the player to clear small obstacles and gain air for trick execution:

```
rb.linearVelocity = new Vector2(currentHorizontalSpeed, jumpForce);
```

The `isJumping` flag prevents double-jumps. It is reset to false in `OnCollisionEnter2D` whenever the player contacts a Ground-tagged or Rock-named surface, allowing jumps from both terrain and rock tops.

5. Creative Implementations (Polish & Game Feel)

5.1. Scoring and Combo System

Beyond simple gem collection, a risk/reward trick system was engineered in `PlayerController.cs`:

- Spin Detection: `totalRotation` accumulates rotation delta each frame via `Mathf.DeltaAngle()`. Every $\geq 290^\circ$ of cumulative rotation while airborne counts as one full spin.
- Trick Reward: Each spin awards $100 \times \text{comboMultiplier}$ points and increments `comboMultiplier`.
- Combo Reset: Landing on any surface resets `comboMultiplier` to 1 and `totalRotation` to 0, encouraging back-to-back aerial combos before touching down.
- Visual Feedback: A `SpinTextPrefab` floats up from the player and fades out, showing "+100 xN" for each completed spin via an animated `Coroutine`.
- Gem Collection: Each `PointGem` awards a configurable `pointValue` (default 10) and plays a collect sound effect.

5.2. Dynamic Weather Effects

A camera-following snowfall system was implemented programmatically in `GameManager.cs` using Unity's Particle System API rather than a static placed prefab:

- A new `GameObject` with `ParticleSystem` is created at runtime during gameplay scenes.
- Configured parameters: 1200 max particles, rate = 150/sec, velocity Y = -5.2 (falling), velocity X = -0.8 (wind drift), shape = `SingleSidedEdge` spanning 40 world units wide.
- `LateUpdate()` repositions the snowfall emitter to match the camera world position every frame, ensuring continuous snow coverage regardless of player progress distance.
- The snowfall is disabled on the Main Menu scene (index 0) to avoid visual clutter.

5.3. High Score Persistence & Leaderboard

A top-5 leaderboard was implemented using Unity's PlayerPrefs system (HighestScore.cs):

- On Game Over or Win, the player's final score is passed to HighestScore.SaveHighScore().
- The method retrieves the existing 5 scores, appends the new score, sorts descending, and saves back the top 5 entries (keys: HighScore0 ... HighScore4).
- The Main Menu High Score panel retrieves and displays these values via ShowHighScores() in MainCamera.cs, reading PlayerPrefs.GetInt("HighScore" + i).

5.4. Audio Feedback System

Audio is layered across gameplay to reinforce game feel:

Audio File	Type	Triggered By
christmas-snow-176839.mp3	BGM	Main Menu background music
the_mountain-8-bit-retro-522443.mp3	BGM	Level gameplay background music
snow-132947.mp3	BGM	Alternative level music
Crash SFX.ogg	SFX	CrashDetector.Crash() — player wipeout
Finish SFX.ogg	SFX	FinishLine trigger — level complete
diamond-found-190255.mp3	SFX	Point.cs — PointGem collected

6. Challenges Faced & Accommodations

■ Slope Mechanics Balance

Challenge: Fine-tuning Rigidbody2D torque against SurfaceEffector2D speed to make the snowboarder feel stable on slopes while remaining responsive for aerial tricks was a significant challenge. Excessive torque caused uncontrollable spinning; insufficient torque made tricks feel unresponsive.

Solution: Iterative playtesting was used to calibrate TorqueAmount, boostSpeed, baseSpeed, and the Rigidbody2D gravity scale until a satisfying balance was achieved. All values are exposed as SerializeField parameters for easy Inspector tuning.

■ Terrain Collider Smoothness

Challenge: Unity Sprite Shape requires a synchronized EdgeCollider2D to function as physics terrain. Misalignment between the visual spline and the collider points caused invisible walls and unexpected player bouncing.

Solution: The Sprite Shape Controller's auto-generated collider was used directly, with manual adjustment of edge points to remove sharp corners that would catch the player collider.

■ Camera-Following Snowfall

Challenge: A static particle emitter placed in the scene stops producing snow once the player moves far from the spawn point, breaking the weather effect illusion.

Solution: Implemented a runtime-created ParticleSystem parented to a custom manager that repositions itself to the camera's world position every LateUpdate(), ensuring persistent snowfall regardless of player distance traveled.

■ Finish Line Trigger Reliability

Challenge: The FinishLine trigger relies on the Player tag being correctly assigned to the root Barry GameObject. If the tag is on a child object instead, the trigger condition collision.gameObject.tag == "Player" fails silently.

Solution: Documented requirement: tag must be set on the root Barry object (not Boarder_Top or Boarder_Bottom). The BoxCollider2D offset and size were adjusted to ensure the trigger volume covers the player's travel path at the flag position.

7. Feature Completion Summary

Requirement	Points	Status	Implementation
Snowboarding Adventure (Player Controls)	20	✓ Implemented	PlayerController.cs — Torque rotation, SurfaceEffector speed control, jump (Space)
Unity Physics Engine (Slope Physics)	15	✓ Implemented	Rigidbody2D + SurfaceEffector2D + Sprite Shape EdgeCollider2D terrain
Obstacle Course & Challenges	20	✓ Implemented	2 Levels, Trees, Rocks (bouncy collider), PointGems, Finish Line
User Interface Design	10	✓ Implemented	Main Menu (Play/Options/HighScore/Quit), In-game HUD (Score/Speed/Distance/Combo)
Collisions & Triggers	15	✓ Implemented	CrashDetector triggers, PointGem triggers, FinishLine trigger, Rock physics
Scoring & Combo System	20	✓ Implemented	Gem points + Spin trick points + Combo multiplier + High Score persistence
Dynamic Weather (Optional)	—	✓ Bonus	Camera-following particle snowfall system (GameManager.cs)
Multiple Levels (Optional)	—	✓ Bonus	Level1 + Level2 with distinct terrain and difficulty
Audio System (Optional)	—	✓ Bonus	BGM + 3 SFX (crash, finish, collect) + AudioSource components

Leaderboard (Optional)	–	✓ Bonus	Top-5 High Score saved via PlayerPrefs, displayed in Main Menu
------------------------	---	---------	--

8. Conclusion

The "Snow Boarder" project successfully satisfies all mandatory Lab 2 requirements and extends beyond them with four creative bonus implementations: dynamic weather, multi-level progression, layered audio, and persistent leaderboard.

The core architecture — Rigidbody2D physics, SurfaceEffector2D slope traversal, Sprite Shape terrain, Cinemachine camera, and TextMeshPro HUD — provides a robust, scalable foundation. The trick scoring system introduces meaningful risk/reward gameplay depth that distinguishes the project from a basic physics demo.

Areas identified for future improvement include: implementing a player lives system for extended runs, adding power-up collectibles (speed boost, invincibility), and fully activating the speed-based score accumulation (currently commented out, pending balance tuning). These additions would complete the full game concept specification without altering the existing architecture.